EX601 Assignment 1: Individual Design Slice

CareFlow (Group 9): Review and Change Triage Priority

Name: Gabriella Piccolo

Part 1: Scope and Requirements Slice

```
Slice name:   Review and change triage priority


In scope:     Nurse views the list of submitted intake forms; nurse reviews the
              intake and assigns a triage priority; nurse changes a patient's
              priority later


Out of scope: Clinician views the queue in priority order; clinician updates
              status after evaluation; patient checks their queue position or
              wait time; patient submits the intake form (this slice assumes the
              form is already submitted); nurse flags long wait times
```

Requirements

- **R1** As a nurse, I can see a list of patients who have submitted their intake forms, so that I can keep track of who has been reviewed and who has not.
- **R2** As a nurse, I can assign a patient's triage priority, so that patients are seen in the order of their medical need.
- **R3** As a nurse, I can change a patient's priority that has already been assigned, so that in the case there is a change in symptoms or emergency, patients are seen in the correct order.

Rules / invariants

- A patient prioritized as 'CRITICAL' is pushed to the top of the list. (R2, R3)
- Only authorized nurses should be able to assign and change patient priority. (R2, R3)
- Changes to triage priority must be recorded. (R3)
- A nurse cannot assign priority to a patient whose intake form is missing required information. (R2)

Part 2: Domain Model

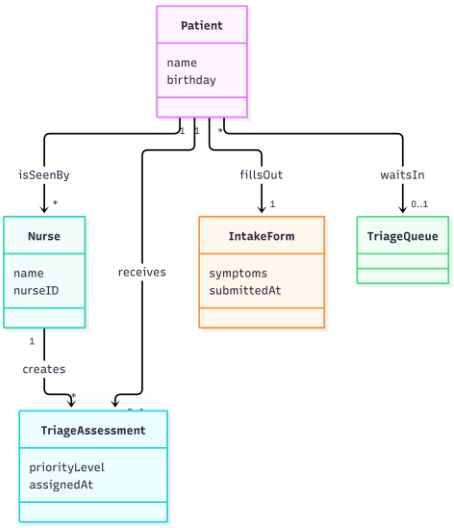
Entities and key attributes

```
Patient:          name, birthday
Nurse:            name, nurseID
IntakeForm:       symptoms, submittedAt
TriageAssessment: priorityLevel, assignedAt
TriageQueue:      (no attributes of its own; it holds waiting patients via its relationship)
```

Relationships (with multiplicity)

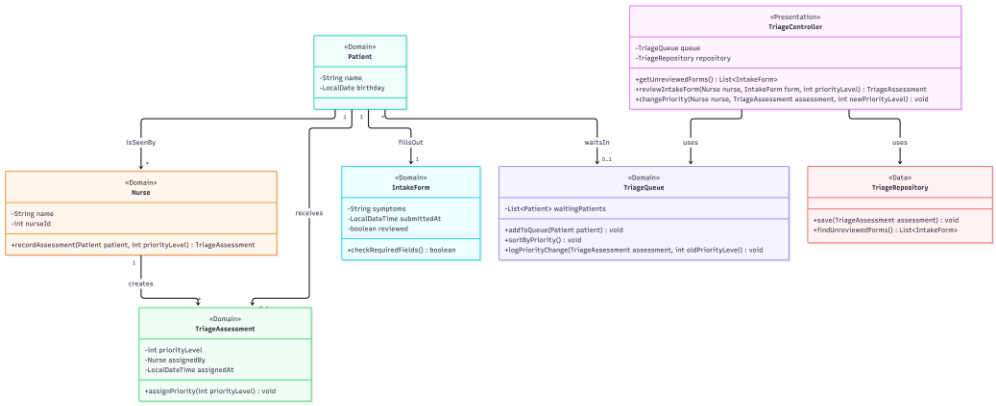
```
Patient "*" --> "*" Nurse : isSeenBy
Nurse "1" --> "*" TriageAssessment : creates
Patient "1" --> "0..1" TriageAssessment : receives
Patient "1" --> "1" IntakeForm : fillsOut
Patient "*" --> "0..1" TriageQueue : waitsIn
```

Domain class diagram



Part 3: Design Model

Design class diagram



Responsibility table

Behavior	Owner class	GRASP question
reviewIntakeForm	TriageController	Controller
recordAssessment	Nurse	Creator
addToQueue	TriageQueue	Information Expert
checkRequiredFields	IntakeForm	Information Expert
sortByPriority	TriageQueue	Information Expert
assignPriority	TriageAssessment	Information Expert
logPriorityChange	TriageQueue	Information Expert
getUnreviewedForms	TriageController	Controller
changePriority	TriageController	Controller

Layer map

```
Presentation: TriageController
Domain:       Nurse, Patient, IntakeForm, TriageQueue, TriageAssessment
Data:         TriageRepository
```

Key design-level signatures (Java)

```
List<IntakeForm>  TriageController.getUnreviewedForms()
TriageAssessment  TriageController.reviewIntakeForm(Nurse nurse, IntakeForm form, int
priorityLevel)
void              TriageController.changePriority(Nurse nurse, TriageAssessment assessment, int
newPriorityLevel)
boolean           IntakeForm.checkRequiredFields()
TriageAssessment  Nurse.recordAssessment(Patient patient, int priorityLevel)
void              TriageAssessment.assignPriority(int priorityLevel)
void              TriageQueue.addToQueue(Patient patient)
void              TriageQueue.sortByPriority()
void              TriageQueue.logPriorityChange(TriageAssessment assessment, int oldPriorityLevel)
void              TriageRepository.save(TriageAssessment assessment)
List<IntakeForm>  TriageRepository.findUnreviewedForms()
```

Part 4: A SOLID Decision, with an ADR

```
Principle in focus: Open/Closed

Decision path:      [x] Applied by design   [ ] Refactor   [ ] Restraint

The likely change:  A clinic adds a rule that bumps up patients who have waited
                    longer than a set time, e.g. if its Emergency Room sees very
                    long wait times.

GRASP question:     Protected Variations (the sorting rule is what varies, so
                    TriageQueue is shielded from it behind an interface)

Code (interface + 2 implementations; priority levels 1-5, 5 = CRITICAL)
```

```
public interface PriorityOrderingStrategy {
    Comparator<Patient> comparator();
}

public class HighestPriorityFirstStrategy implements PriorityOrderingStrategy {
    public Comparator<Patient> comparator() {
        return Comparator.<Patient>comparingInt(
            p -> p.getAssessment().getPriorityLevel()).reversed();
    }
}

// Long waits move up one level, but never to CRITICAL, so rule 1 still holds
public class LongWaitBoostStrategy implements PriorityOrderingStrategy {
    private static final int CRITICAL = 5;
    private final Duration maxWait;

    public LongWaitBoostStrategy(Duration maxWait) { this.maxWait = maxWait; }

    public Comparator<Patient> comparator() {
```

```
        return Comparator.<Patient>comparingInt(this::effectivePriority).reversed();
    }

    private int effectivePriority(Patient p) {
        int level = p.getAssessment().getPriorityLevel();
        Duration waited = Duration.between(
            p.getIntakeForm().getSubmittedAt(), LocalDateTime.now());
        boolean longWait = waited.compareTo(maxWait) > 0;
        return (longWait && level < CRITICAL) ? Math.min(level + 1, CRITICAL - 1) : level;
    }
}

public class TriageQueue {
    private final List<Patient> waitingPatients = new ArrayList<>();
    private final PriorityOrderingStrategy ordering;

    public TriageQueue(PriorityOrderingStrategy ordering) { this.ordering = ordering; }

    public void addToQueue(Patient patient) {
        waitingPatients.add(patient);
        sortByPriority();
    }

    public void sortByPriority() {
        waitingPatients.sort(ordering.comparator());
    }
}
```

ADR:

```
# ADR-001: Move the triage sorting rule behind an interface
Status: Accepted
Date: 2026-10-04

## Context
A clinic might need to change how it sorts patients. For example, a rule that
bumps up patients after they have waited a certain amount of time, if the
Emergency Room is seeing very long waits. Today the sorting rule is written
inside TriageQueue, so any new rule would mean editing that class.

## Decision
Apply Open/Closed. TriageQueue depends on a PriorityOrderingStrategy interface.
Today's rule (HighestPriorityFirstStrategy) and the long-wait rule
(LongWaitBoostStrategy) are two implementations. A new rule is a new class,
not an edit to TriageQueue.

## Consequences
+ New sorting rules can be added without changing working code in TriageQueue,
  and rule 1 (CRITICAL on top) stays in one tested place per strategy.
- One more interface and class for a teammate to read, and a level of
  indirection when following how the queue is sorted.
```

Part 5: The AI Step

Prompt used: Can you take this class “TriageQueue” and make it more SOLID?
Assistant output:

```
public class TriageQueue {
    private List<Patient> waitingPatients = new ArrayList<>();
```

```
    public void addToQueue(Patient patient) {
        waitingPatients.add(patient);
        sortByPriority();
    }

    public void sortByPriority() {
        waitingPatients.sort((a, b) ->
            b.getAssessment().getPriorityLevel() - a.getAssessment().getPriorityLevel());
    }

    public void logPriorityChange(TriageAssessment assessment) {
        System.out.println(assessment.getPatient().getName()
            + " changed to priority " + assessment.getPriorityLevel()
            + " by " + assessment.getAssignedBy().getName());
    }
}

// ---- Change A: ordering pulled out behind an interface ----
public interface PriorityOrderingStrategy {
    Comparator<Patient> comparator();
}

public class HighestPriorityFirstStrategy implements PriorityOrderingStrategy {
    public Comparator<Patient> comparator() {
        return (a, b) -> b.getAssessment().getPriorityLevel()
                        - a.getAssessment().getPriorityLevel();
    }
}

// ---- Change B: logging moved to its own audit logger ----
public interface PriorityChangeLogger {
    void logPriorityChange(TriageAssessment assessment);
}

public class ConsoleAuditLogger implements PriorityChangeLogger {
    public void logPriorityChange(TriageAssessment assessment) {
        System.out.println(assessment.getPatient().getName()
            + " changed to priority " + assessment.getPriorityLevel()
            + " by " + assessment.getAssignedBy().getName());
    }
}

// ---- Change C: storage hidden behind a repository ----
public interface PatientQueueRepository {
    void save(Patient patient);
    List<Patient> findAll();
}

public class InMemoryPatientQueueRepository implements PatientQueueRepository {
    private final List<Patient> patients = new ArrayList<>();
    public void save(Patient patient) { patients.add(patient); }
    public List<Patient> findAll() { return new ArrayList<>(patients); }
}

// ---- Change D: an interface for the queue itself ----
public interface Queue {
    void addToQueue(Patient patient);
    List<Patient> getOrderedPatients();
}
```

```
// ---- Change E: a factory to build the queue ----
public class TriageQueueFactory {
    public static Queue create() {
        return new TriageQueue(
            new InMemoryPatientQueueRepository(),
            new HighestPriorityFirstStrategy(),
            new ConsoleAuditLogger());
    }
}

// ---- The refactored TriageQueue ----
public class TriageQueue implements Queue {
    private final PatientQueueRepository repository;
    private final PriorityOrderingStrategy ordering;
    private final PriorityChangeLogger logger;

    public TriageQueue(PatientQueueRepository repository,
                       PriorityOrderingStrategy ordering,
                       PriorityChangeLogger logger) {
        this.repository = repository;
        this.ordering = ordering;
        this.logger = logger;
    }

    public void addToQueue(Patient patient) {
        repository.save(patient);
    }

    public List<Patient> getOrderedPatients() {
        List<Patient> patients = repository.findAll();
        patients.sort(ordering.comparator());
        return patients;
    }

    public void recordPriorityChange(TriageAssessment assessment) {
        logger.logPriorityChange(assessment);
    }
}
```

What the assistant said about each change

- **A. PriorityOrderingStrategy:** "Open/Closed: new ordering rules can be added without modifying TriageQueue."
- **B. PriorityChangeLogger:** "Single Responsibility: logging is a separate concern. Dependency Inversion: the queue depends on an abstraction, so logging can go to a file or database later."
- **C. PatientQueueRepository:** "Dependency Inversion: storage can be swapped (for example, to a database) without touching queue logic."
- **D. Queue interface:** "Dependency Inversion / Liskov: clients depend on an abstraction, so alternative queue implementations can be substituted."
- **E. TriageQueueFactory:** "Centralizes object creation so wiring can change in one place."

The assistant ended with: "The class now follows SOLID and is easy to extend."

My Judgement:

The assistant applied several principles at once: Open/Closed (moving the sorting rule into PriorityOrderingStrategy), Single Responsibility (moving logging into PriorityChangeLogger), and Dependency Inversion (PatientQueueRepository and a Queue interface). It also added a TriageQueueFactory. For a class that started with three methods and no interfaces, it added four new interfaces and four new classes.

I would accept PriorityOrderingStrategy. A clinic may realistically need to change how it sorts patients, for example by adding a rule that bumps up patients after a long wait if its Emergency Room is seeing very long wait times. With the interface, that rule becomes a new class instead of an edit to TriageQueue, which is the decision recorded in ADR-001.

I would push back on the Queue interface. CareFlow will only ever have one triage queue, so the interface has only one realistic implementation. The one thing that might vary, the sorting rule, is already handled by PriorityOrderingStrategy. The interface is more for a teammate to read, and every change must be made in two places (TriageQueue and Queue) for no return. TriageQueueFactory has the same problem, since it only ever builds one kind of queue. Before adding an interface, I ask whether the addition is justifiable and realistic given the system's goals.

Part 6: Reflection and Team Handoff

Reflection

Open/Closed was the principle most at work in my slice. The part of CareFlow most likely to change is how the queue orders patients, not the queue itself or how patients and nurses relate. Pulling the sorting rule out of TriageQueue and behind PriorityOrderingStrategy means a new rule can be added as a new class without editing code that already works.

I know this decision was warranted, not over-engineering, because it is tied to a specific, realistic change: a rule that moves patients up after a long wait, which a clinic would want if its Emergency Room is seeing very long wait times. That rule is concrete enough that I wrote it as a second implementation, LongWaitBoostStrategy, so the interface has two real uses today rather than one imagined one. By contrast, I did not add a Queue interface or a factory, because CareFlow only needs one triage queue and no second case exists. The test I used for every abstraction was the same: can I name the change it protects against? If I can, the interface earns its place. If I cannot, it is just more for a teammate to read.

Team handoff

- Slice I covered: Review and change triage priority
- Strongest piece to feed Checkpoint 1: SOLID decision, because PriorityOrderingStrategy gives the team one place to plug in new sorting rules (such as the long-wait flagging that was out of my scope) when we assemble the slices.